

PRACTICAL WEEK 9

TASK1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    // Open source file for reading
    src = open(argv[1], O_RDONLY);

    if (src < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open/create destination file for writing
    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0)
    {
        perror("Error opening destination file");
        close(src);
        return 1;
    }
```

```
        perror("Error opening destination file");
        close(src);
        return 1;
    }

    // Read from source and write to destination
    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("Error writing file");
            close(src);
            close(dest);
            return 1;
        }
    }

    if (bytes_read < 0)
    {
        perror("Error reading file");
    }

    // Demonstrate lseek()
    lseek(src, 0, SEEK_SET);

    close(src);
    close(dest);

    printf("File copied successfully using low-level I/O.\n");

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/  ×  +  ▾  -  □
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    // Open source file
    src = fopen(argv[1], "rb");

    if (src == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open destination file
    dest = fopen(argv[2], "wb");

    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }
}
```

```
jhansi_priya@Jhansi-Priya: ~/  ×  +  ▾  -  □
GNU nano 7.2 task2.c *
    if (src == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open destination file
    dest = fopen(argv[2], "wb");

    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }

    // Read and write using standard library functions
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
        {
            perror("Error writing file");
            fclose(src);
            fclose(dest);
            return 1;
        }
    }

    fclose(src);
    fclose(dest);

    printf("File copied successfully using standard library I/O.\n");

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ x + v
jhansi_priya@Jhansi-Priya:~$ mkdir Practical9
jhansi_priya@Jhansi-Priya:~$ cd Practical9
jhansi_priya@Jhansi-Priya:~/Practical9$ echo "This is a sample file for tes
ting file copy operations in Linux." > input.txt
jhansi_priya@Jhansi-Priya:~/Practical9$ cat input.txt
This is a sample file for testing file copy operations in Linux.
jhansi_priya@Jhansi-Priya:~/Practical9$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Practical9$ gcc task1.c -o task1
jhansi_priya@Jhansi-Priya:~/Practical9$ ./task1
Usage: ./task1 <source> <destination>
jhansi_priya@Jhansi-Priya:~/Practical9$ ls
input.txt task1 task1.c
jhansi_priya@Jhansi-Priya:~/Practical9$ ./task1 input.txt output
_low.txt
File copied successfully using low-level I/O.
jhansi_priya@Jhansi-Priya:~/Practical9$ cat output_low.txt
This is a sample file for testing file copy operations in Linux.
jhansi_priya@Jhansi-Priya:~/Practical9$ cmp input.txt output_low
.txt
jhansi_priya@Jhansi-Priya:~/Practical9$ nano task2.c
jhansi_priya@Jhansi-Priya:~/Practical9$ gcc task2.c -o task2
jhansi_priya@Jhansi-Priya:~/Practical9$ ./task2 input.txt output_lib.txt
File copied successfully using standard library I/O.
```

TASK2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
GNU nano 7.2 redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int input_fd, output_fd;

    // Open input file
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening input file");
        return 1;
    }

    // Redirect standard input to input.txt
    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input");
        return 1;
    }

    close(input_fd);

    // Open output file
    output_fd = open("output.txt",
O_WRONLY | O_CREAT | O_TRUNC,
0644);

    if (output_fd < 0)
    {
        perror("Error opening output file");
        return 1;
    }

    // Redirect standard output to output.txt
    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        return 1;
    }

    close(output_fd);

    // This output goes into output.txt
    printf("This line is redirected to output.txt.\n");

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ▾
GNU nano 7.2 read.c *
#include <stdio.h>

int main()
{
    char str[100];

    scanf("%99s", str);

    printf("You entered: %s\n", str);

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ▾ - □ ×
jhansi_priya@Jhansi-Priya:~$ cd Practical9
jhansi_priya@Jhansi-Priya:~/Practical9$ nano redirect.c
jhansi_priya@Jhansi-Priya:~/Practical9$ gcc redirect.c -o redirect
jhansi_priya@Jhansi-Priya:~/Practical9$ ./redirect
jhansi_priya@Jhansi-Priya:~/Practical9$ cat output.txt
This line is redirected to output.txt.
jhansi_priya@Jhansi-Priya:~/Practical9$ ls > files.txt
jhansi_priya@Jhansi-Priya:~/Practical9$ cat files.txt
files.txt
input.txt
output.txt
output_lib.txt
output_low.txt
redirect
redirect.c
task1
task1.c
task2
task2.c
jhansi_priya@Jhansi-Priya:~/Practical9$ nano read.c
jhansi_priya@Jhansi-Priya:~/Practical9$ gcc read.c -o read
jhansi_priya@Jhansi-Priya:~/Practical9$ ./read
echo "Hello" > input.txt
You entered: echo
jhansi_priya@Jhansi-Priya:~/Practical9$ ./read < input.txt
You entered: This
jhansi_priya@Jhansi-Priya:~/Practical9$ |
```


